

Transformer on Jac + Jac on GPU

Executable graphs, named cursors, physical layouts and measured outcomes

Part I: Transformer on Jac

Preserved module-level model.
New operation-level dataflow graph.
CPU/CUDA correctness and analysis.

Part II: Jac on GPU

Named cursor execution.
Corrected A-then-B BFS packing.
Full 3000-cubed MatMul and profiling.

Read the evidence at the right level

Claim	Evidence	Boundary
Transformer graph works	CPU/CUDA differential tests	Random weights; inference only
New dataflow analysis works	Real graph edits + invalid graph tests	Concrete shapes; no alias analysis
Random layout is slower at 3000 ³	Paired Nsight kernel measurements	Structured int64 fixture
New BFS beats Current	Not established	Small differences change direction
Cache/coalescing explains speed	Not established	Nsight Compute permission denied

I. Transformer on Jac

Two independent examples; the original 80 files remain unchanged

A graph should express executable dependencies, then support analysis.

Module-level nodes explain the architecture. Operation-level nodes expose tensor producers, consumers and intermediate values.

Neither representation alone proves an advantage over PyTorch FX / export.

The preserved module-level model

Graph and execution

18 computation nodes, 22 data edges.
Weights belong to concrete module nodes.
A host Jac walker schedules ready nodes.

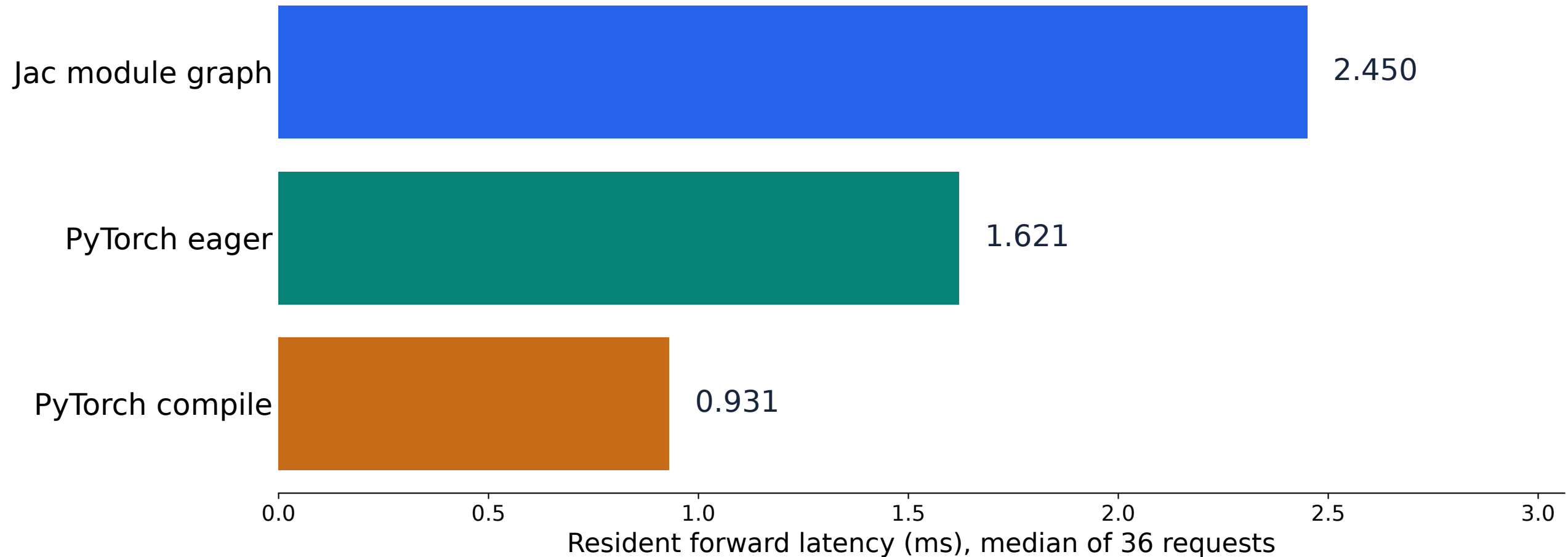
Tensor CPU / CUDA path

PyTorch tensor primitives execute math.
Weights and activations stay on device.
Edges pass tensor references.
No per-module download or sync.

One encoder + one decoder; post-norm, causal attention and cross-attention.

Existing Transformer: GPU latency baseline

Medium float32: d=256, heads=8, FF=1024, vocabulary=1024; lengths 128 / 96



CPU medians: 6.287 / 5.566 / 5.089 ms in the same method order.

Equivalent weights and observables; no Jac speed advantage measured.

Profiling the preserved implementation

Independent profiling runs; these durations are not a decomposition of forward latency

Method	Kernels / request	Profile kernel sum / request (ms)
Jac graph	123	0.4123
Eager	123	0.4134
Compile	53	0.2609

Jac and eager execute the same number of math kernels. Compile reduces kernel count.
Host traversal / scheduling costs are observed in cProfile and handle-replay tests.
Profile instrumentation and submission gaps prevent a strict causal subtraction.

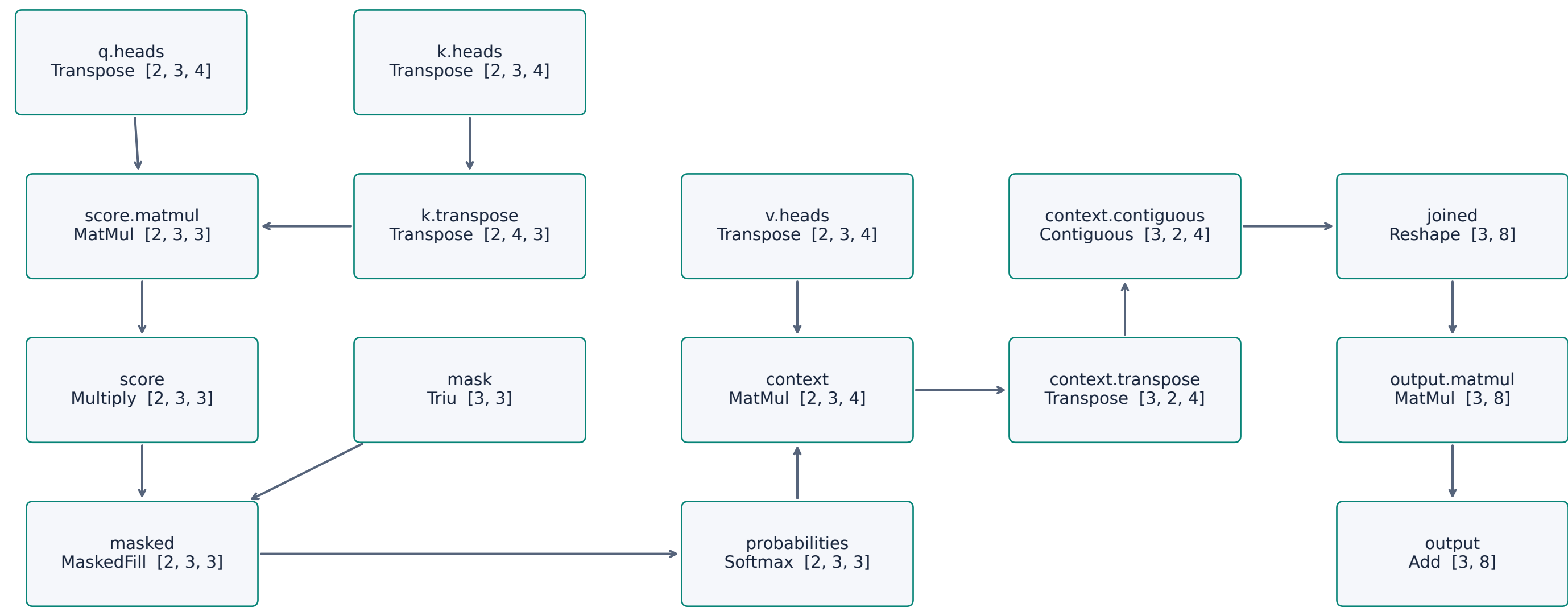
New example: operation-level dataflow

Graph property	Measured / exported value
Nodes / tensor edges	200 / 224
Parameter nodes	46
Independently checked intermediates	137
Execution contract	Producer output -> named consumer port
Analysis / execution	Separate Jac walkers

Linear, Attention, FFN and LayerNorm are build-time expansions, not runtime black boxes.

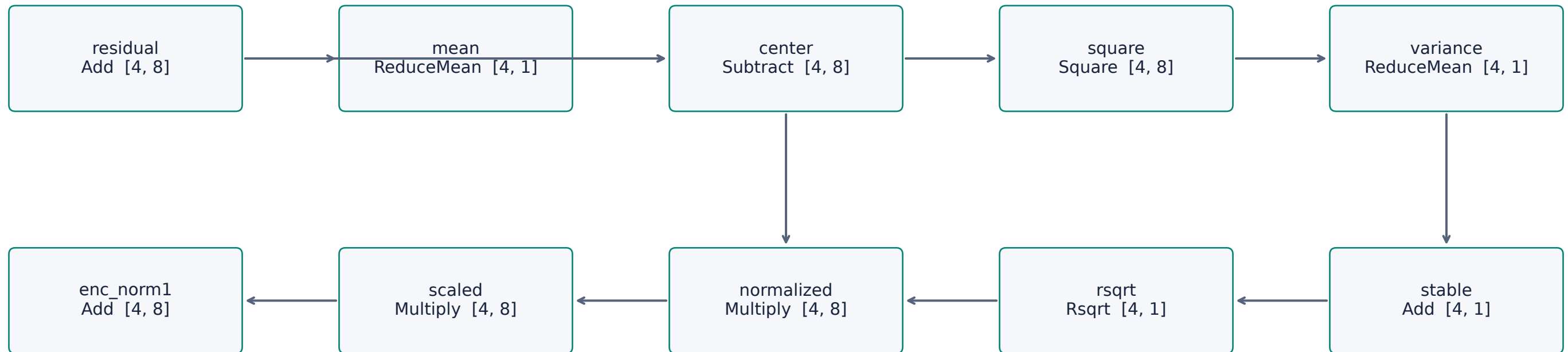
Actual decoder attention: selected primitive nodes

Direct edges only. Q/K/V projection subgraphs and some boundary inputs are omitted here.



Actual LayerNorm expansion

Selected exported nodes and their direct dependencies; gamma / beta / epsilon are parameter or constant inputs



Residual Add -> mean / centering -> variance -> inverse sqrt -> gamma / beta affine.

Shape and dtype labels come from the actual analysis, not a handwritten architecture diagram.

What the analysis walker computes

Analysis	Implementation / contract
Dependencies	Enumerate real edges; check port producers and roots
Metadata propagation	Shape, dtype, device; meta tensor operation rules
Topological order	Visit only after every input is ready
Invalid graphs	Reject missing/duplicate inputs, cycles and bad shapes
Plan invalidation	Fingerprint edges, attributes and parameter metadata
Liveness	Compute last consumers; clear values after last use

Correctness and graph mutation evidence

8 CPU/CUDA cases: two configurations x two precisions x two devices.
137 intermediate results checked against independent equations.
Maximum CPU/CUDA difference: float64 1.42e-14; float32 6.68e-06.

Positive graph edits

Rewire a residual input.
Insert Constant + Multiply in logits.
The execution walker stays unchanged.

Negative checks

Missing encoder-memory input.
11 additional invalid graph cases.
Changed shape / attributes invalidate plan.

Earlier dataflow example: scope and limitations

Established

Real primitive graph and parameter nodes.
Metadata, topology and last-use analysis.
CPU/CUDA execution and graph-edit tests.
Original 80 files hash-preserved.

Not established

No new-model performance comparison.
No FX/export advantage demonstrated.
No fusion, symbolic shapes or training.
Logical liveness bytes are not allocator
memory; aliases are not analyzed.

Reusable runtime library: a third sibling example

The model builds architecture from standard primitive node types and real tensor edges. Parameter nodes own weights. One public Compute walker executes the graph. Readiness, dependency validation, metadata and lifetime management live in the library.

Preserved examples

transformer_graph: 80 unchanged files.
transformer_dataflow: 74 unchanged files.

New implementation

transformer_runtime uses the library.
Exact type identity defines supported operation semantics.

Implemented path: Jac graph to FX to Inductor

1. Read actual Jac nodes, port-labelled edges and the dependency plan.
2. Construct an FX GraphModule directly; bind node-owned parameters.
3. Execute FX eagerly, or apply `torch.compile` with `backend="inductor"`.
4. Guard graph and input metadata; refresh same-metadata parameter values.

The current Inductor entry is `torch.compile(fullgraph=True, dynamic=False)`.
It still uses the PyTorch compilation frontend on the constructed GraphModule.
This is library/runtime lowering; no Jac compiler recognition pass is implemented.

Runtime Transformer: CPU and CUDA correctness

Eight cases; small / medium, float32 / float64, CPU / CUDA

Device	Configuration	FX vs reference	Inductor vs reference
cpu	small_float64	0.00e+00	2.66e-15
cpu	small_float32	2.86e-06	1.67e-06
cpu	medium_float64	0.00e+00	1.60e-14
cpu	medium_float32	6.68e-06	5.72e-06
cuda_0	small_float64	0.00e+00	4.44e-15
cuda_0	small_float32	1.19e-06	1.91e-06
cuda_0	medium_float64	1.24e-14	1.20e-14
cuda_0	medium_float32	8.58e-06	6.08e-06

All cases pass. Compiled forward uses fullgraph=True; tested forwards have no graph break.
Repeated calls, parameter refresh and graph-edit checks are recorded. No new timing comparison.

Cond: extract shared work and two lazy branches

Three graph ports: condition, true_result, false_result.

C = condition dependency slice; T / F = true / false result slices.

Before selection: compute C and shared T intersection F dependencies.

Inside each branch: compute its exclusive dependency region.

Boundary values and weights become explicit operands.

FX contains `torch.ops.higher_order.cond(predicate, true_graph, false_graph, operands)`.

Branches are extracted FX modules. Only the selected exclusive region executes.

Cond validation and current restrictions

Device	Dtype	Walker error	FX error	Inductor error	Invalid checks
cpu	float64	0.0	0.0	0.0	14
cuda_0	float64	0.0	0.0	0.0	14

Positive / negative predicates, shared fanout, identical branch roots and weight refresh pass.
Selected-only walker traces and unselected out-of-range Gather validate lazy execution.
Supported: one non-nested Cond in a pure acyclic graph; scalar bool predicate.
Nested / multiple Cond, raw cycles, incompatible outputs and escaping branch values are rejected.
Compute walker reads predicate on host; compiled path keeps structured Cond. No speed claim.

II. Jac on GPU

One integer per graph node; joint cursor arrival and predictable traversal structure

Independent FIFO per cursor and lane.

A round consumes one fresh arrival from every cursor.

Current positions remain stable during the joint ability.

Packing changes storage indices, not execution events.

A fixed number of cursors is declared at compile time; no special two-cursor ABI.

MatMul: two graph paths per output

Cursor a: A row

$A[i,0] \rightarrow A[i,1] \rightarrow \dots \rightarrow A[i,K-1]$

Follow RowNext.

Shared by every output in row i .

Cursor b: B column

$B[0,j] \rightarrow B[1,j] \rightarrow \dots \rightarrow B[K-1,j]$

Follow ColumnNext.

Shared by every output in column j .

One lane accumulates $C[i,j] = \text{sum over } k \text{ of } A[i,k] * B[k,j]$.

Corrected Predicted: all A, then all B

Multi-source BFS per cursor; concatenate in declaration order; shared identities appear once

Physical slots	Actual 4 x 4 Predicted identity order
0-3	A00 A10 A20 A30
4-7	A01 A11 A21 A31
8-11	A02 A12 A22 A32
12-15	A03 A13 A23 A33
16-19	B00 B01 B02 B03
20-23	B10 B11 B12 B13
24-27	B20 B21 B22 B23
28-31	B30 B31 B32 B33

The earlier rank-interleaved implementation was removed. Historical measurements are labelled.

Smaller MatMul: no stable new BFS gain

Pure resident kernel duration from independent Nsight Systems replay

M x K x N	Current (us)	Predicted (us)	Random (us)
16 x 32 x 24	47.297	47.329	44.400
32 x 64 x 48	93.024	92.736	98.080
64 x 128 x 64, first run	237.266	259.522	238.674

Large-case controlled replay changed the ranking.
Do not interpret a single median as a stable win or regression.
Every layout preserved exact integer outputs and traversal semantics.

Address sectors: theory is not hardware traffic

64 x 128 x 64; median distinct 32-byte sectors per logical warp load

Logical load	Current	Predicted	Random 928
A value	1	1	1
B value	9	8	32
B tag	9	8	32
B CSR begin / end	9	9	32
B CSR target	8	8	32

Exact packed addresses, per load / warp / round. Assumed aligned arena.
No cache state, backend instruction optimization or actual transaction count.
Nsight Compute counters remain inaccessible: ERR_NVGPUCTRPERM.

Full 3000 x 3000 x 3000: actually executed

Scale / resource	Measured fact
Real input graph	18,000,000 nodes; 17,994,000 edges
Outputs / arithmetic	9,000,000 outputs; 27 billion MACs per layout
Batching	32768 lanes; 275 launches per complete result
Readonly resident graph	0.671 GiB; one upload per layout
CPU graph build / peak RSS	140.80 s / 27.78 GiB full-run peak
Complete validations	Current, Predicted, Random seeds 928 / 929 / 930

Full-result validation: five matching outputs

Structured int64 weights: $A[i,k]=i+1$ and $B[k,j]=j+1$

Independent exact oracle for every output: $C[i,j] = 3000 * (i+1) * (j+1)$.

All five complete result files contain 9,000,000 outputs and share one SHA256.

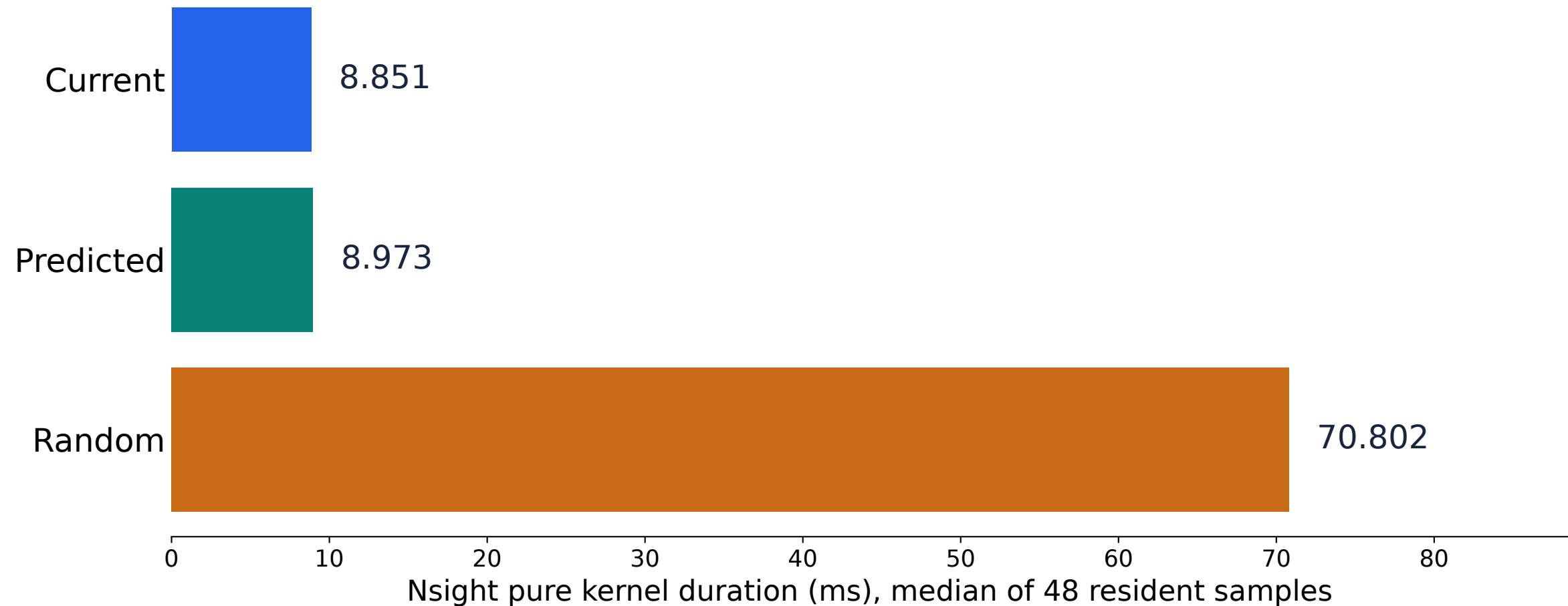
All device status values were zero.

Output Scalar nodes are materialized per batch, saved and released. We do not retain nine million CPU output nodes at once.

These results do not represent arbitrary dense-data or floating-point GEMM performance.

3000-cubed: paired pure kernel measurements

Complete packed graph; each sample computes 32768 outputs, not the whole result



Ordinary event medians: 8.907 / 8.879 / 70.319 ms.

Random is about 8x slower. Current and Predicted remain close; no stable BFS advantage.

Large-run cost and cleanup limitations

Layout	Pack / permutation wall (s)	Full execution + validation / save (s)
Current	328.52	10.72
Predicted	266.74	10.76
Random 928	78.71	27.78
Random 929	77.10	27.80
Random 930	79.81	31.08

Random reuses Current pack. CPU tests overlapped pack: resource facts, not isolated CPU baselines. After all outputs were saved, finalization GC continued; sampled hotspot was gc_collect_main. That finished experiment was terminated (exit 143). This is not a natural-exit end-to-end benchmark.

Measurement environments and contracts

Experiment	Environment	What was measured
Module Transformer	RTX 3090; driver 595.71.05 Python 3.13.7; Torch 2.8.0	2026-10-06; float32 / float64 Host-driven tensor inference
Primitive Transformer	CPU and CUDA; Torch 2.8.0	Correctness and graph analysis No new speed comparison
Large Jac MatMul	RTX 3090; driver 580.65.06 Python 3.12.11; LLVM 20.1.8	2026-10-07; structured int64 Generated Jac walker kernel

Compare methods within each experiment. Cross-date timings and tensor versus scalar workloads do not establish a shared speed ranking.

The module compile benchmark uses fixed shapes and an existing persistent cache.

Pure kernel timings exclude graph construction, packing, transfers and host scheduling.

Conclusions supported by the current evidence

Transformer: executable module and primitive graphs work on CPU/CUDA.
Dataflow analysis, reusable runtime, FX/Inductor execution and lazy Cond are validated.
A speed or compiler-cost advantage over FX/export has not been demonstrated.

Jac GPU: complete A-then-B packing works, including a full 3000-cubed result.
Ordered layouts beat random strongly at that scale. New BFS does not beat Current reliably.
Hardware counter permission is the remaining gap for a causal memory explanation.

Evidence and reproduction

Transformer

- jac/examples/transformer_graph/README.md
- jac/examples/transformer_dataflow/README.md
- jac/examples/transformer_runtime/README.md
- jac/jaclang/lib/tensor_graph/README.md
- Example results/ directories, graph exports and validation summaries

Jac GPU

- docs/design/gpu-cursor-concatenation-results.md
- docs/design/gpu-large-matmul-3000-results.md
- scripts/gpu_walker/large_matmul/validation/

PDF generator and source manifest accompany this deck.